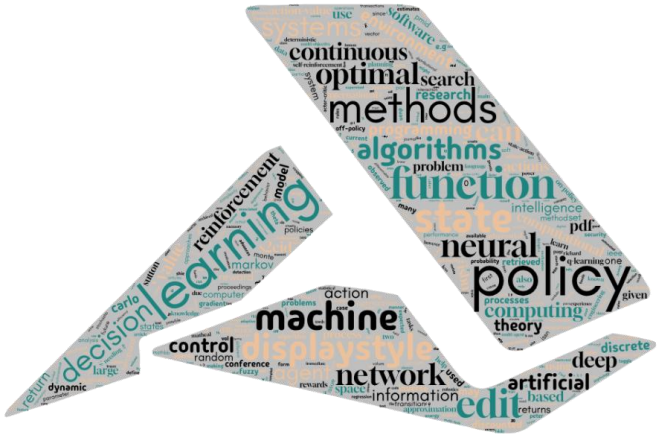


Reinforcement Learning Algorithms and Applications

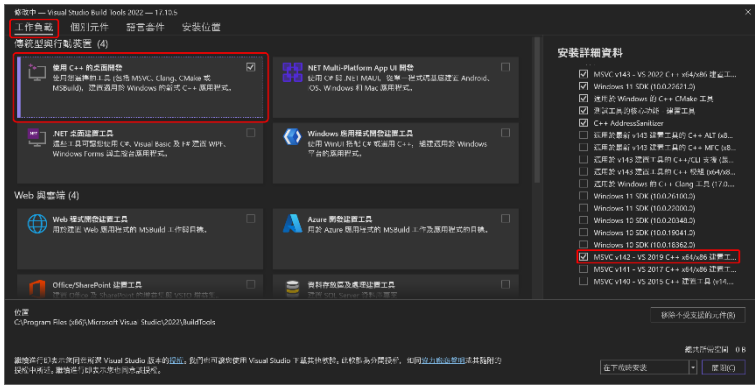
Chapter 1: Epsilon-Greedy Policy for Multi-Armed Bandit

Environmental Setting

- Anaconda Archive
  - https://repo.anaconda.com/archive/
  - Anaconda3-2024.10-1-Windows-x86_64.exe
- Create Environment
  - conda create -n rl python=3.12
  - conda env list
  - conda activate rl
- Leave Environment
  - conda deactivate
- Delete Environment
  - conda remove --name rl --all

Environmental Setting

- Install Pytorch
  - RTX3080
    - pip install torch==2.2.2+cu121 torchvision==0.17.2+cu121 --index-url https://download.pytorch.org/whl/cu121
  - RTX5080
    - pip install torch==2.7.0 torchvision==0.22.0 torchaudio==2.7.0 --index-url https://download.pytorch.org/whl/cu128
- Install Basic libraries
  - pip install scikit-learn pandas
  - conda install matplotlib spyder
- Install gymnasium
  - pip3 install gymnasium[classic_control]
- Install Microsoft C++ Build Tools
  - https://visualstudio.microsoft.com/visual-cpp-build-tools/

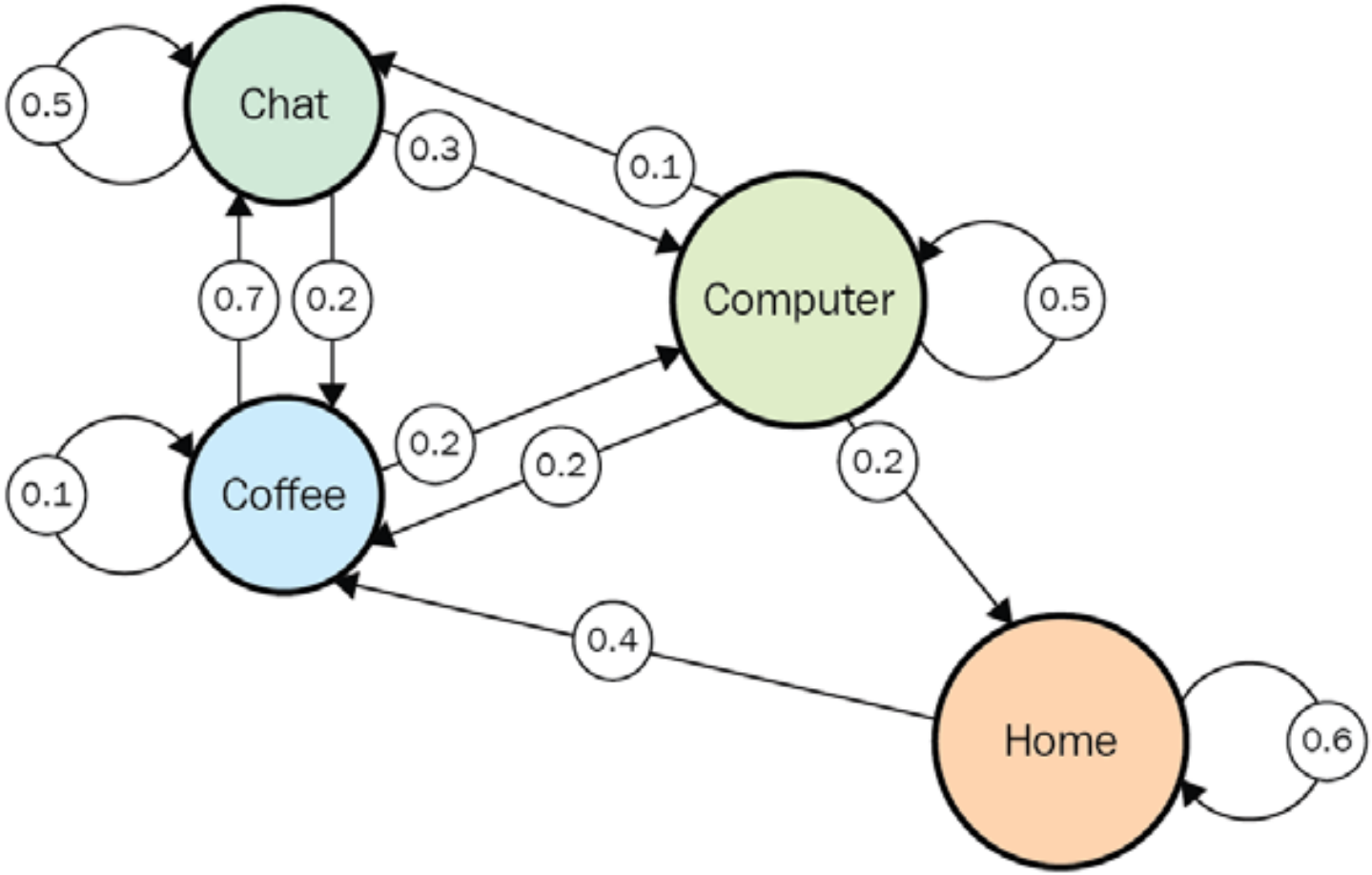


Environmental Setting

- Install gym-super-mario-bros
  - pip install gym-super-mario-bros==7.3.0
  - If get error：ValueError: not enough values to unpack (expected 5, got 4)
    - pip install gym==0.23 nes_py==8.1.8
- Install OpenCV
  - pip install opencv-python==4.8.0.76
- Install Stable Baselines3 (SB3)
  - pip install stable_baselines3==2.0.0
- Reinstall numpy
  - pip install "numpy<2.0"

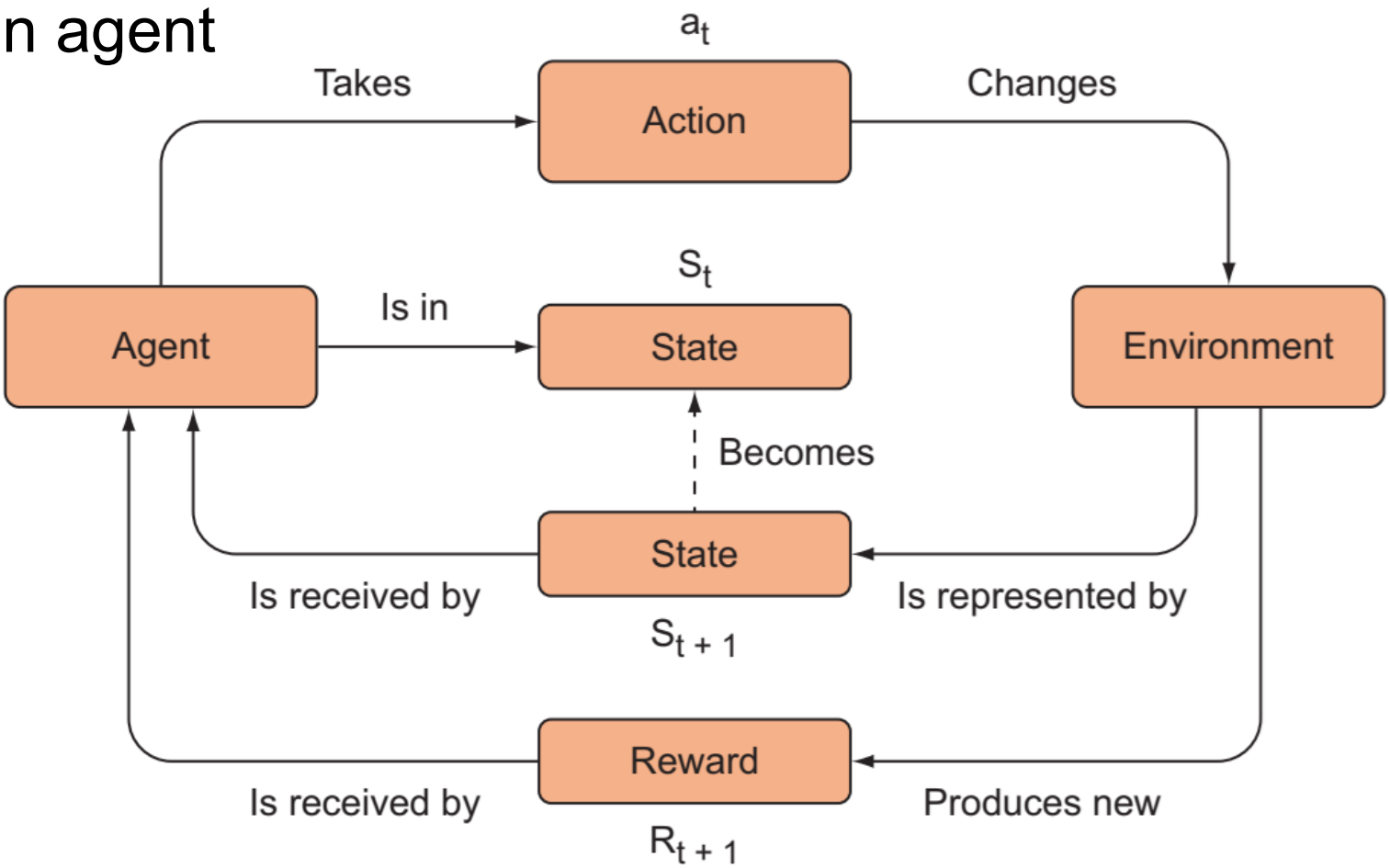
Markov Decision Processes

- The state transition graph with transition probabilities



Reinforcement Learning

- The standard reinforcement learning model in which an agent



Multi-armed bandit

- In probability theory and machine learning, the multi-armed bandit problem (sometimes called the *K* or *N*-armed bandit problem) is a problem in which a decision maker iteratively selects one of multiple fixed choices (i.e., arms or actions) when the properties of each choice are only partially known at the time of allocation, and may become better understood as time passes.
- A fundamental aspect of bandit problems is that choosing an arm does not affect the properties of the arm or other arms.

Introduction for the Game

- Problem Setup
  - K Arms: Each arm has an unknown reward distribution
  - Goal: Maximize cumulative reward
  - Challenge: Unknown true reward expectations for each arm
  - Constraint: Can only choose one arm at a time
- Exploration vs Exploitation
  - Exploration: Try unknown or less-tried options to gather information
  - Exploitation: Choose currently best-known option for immediate reward
  - Dilemma: Over-exploration may miss known good options
  - Risk: Over-exploitation may miss better alternatives
- Real Applications: Ad placement, recommendation systems, clinical trials, investment portfolios, and other domains face similar exploration-exploitation tradeoffs

Epsilon-Greedy Strategy

- **Core Idea:** Explore randomly with probability ε, exploit current best option with probability $(1-\varepsilon)$
- **Initialize:** Set estimated value $Q(a) = 0$, selection count $N(a) = 0$ for each arm
- **Action Selection:** Generate random number, if $< \varepsilon$ choose randomly, else choose argmax $Q(a)$
- **Receive Reward:** Execute selected action, observe reward value
- **Update Estimate:** Use incremental update formula to revise value estimate
- **Value Estimation Update Formula**
  - $Q_{t+1}(a) = Q_t(a) + {}^{1}/_{n}\,[R_t - Q_t(a)]$
    where R_t is the tth reward received, n is the number of times this arm was selected

Epsilon-Greedy Strategy

- **Epsilon Parameter Impact**
  - **ε = 0:** Pure exploitation, may get stuck in suboptimal solutions
  - **ε = 0.01:** Minimal exploration, good balance
  - **ε = 0.1:** Moderate exploration, suitable for most cases
  - **ε = 0.3:** High exploration, fast learning but low efficiency
- **Algorithm Characteristics**
  - **Simple Implementation:** Clear logic, concise code
  - **Parameter Sensitive:** ε value needs problem-specific tuning
  - **Asymptotically Optimal:** Finds optimal solution in the long run
  - **Fixed Exploration Rate:** Doesn't adapt exploration over time

Experimental Design & Results Analysis

- **Experimental Parameters**
  - **Number of Arms:** $K = 10$
  - **Steps:** 2000 steps (single run), 100 repeated experiments
  - **Reward Distribution:** Normal Distribution (average = true_reward, variance $\sigma^2 = 1$)
  - **Evaluation Metrics:** Average reward, cumulative regret, optimal arm selection rate
- **Visualization Components**
  - **Action Selection Frequency:** Distribution of selections across arms
  - **Value Estimation Accuracy:** True values vs estimated values comparison
  - **Action Sequence:** Selection trajectory over recent 1000 steps
  - **Optimal Arm Selection Rate:** Trend over time

Reference

- M. Lapan, *Deep Reinforcement Learning Hands-On*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.
- A. Zai and B. Brown, *Deep Reinforcement Learning in Action*. Shelter Island, NY: Manning Publications, 2020.
- S. Ravichandiran, *Deep Reinforcement Learning with Python*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.
- https://en.wikipedia.org/wiki/Multi-armed_bandit
- https://ithelp.ithome.com.tw/articles/10343384